

Friedrich-Alexander-Universität

Erlangen-Nürnberg (FAU)

NHR@FAU

Assignment 6: OpenMP

Programming Techniques for Supercomputers (PTfS)

Muhammad Omar
Student IdM: to37gulo
Student HPC-Account: ptfs356v

Gehan Younes
Student IdM: qi39dyry

Ahmed Mohamed
Student IdM: gi10dafy

June 18, 2026

Q1: Machine topology – Sapphire Rapids node (15 crd)

Determined with `likwid-topology -g` on a `spr1tb` node (2× Intel Xeon Platinum 8470, "Sapphire Rapids").

(a) Cores per node (5 crd). The node has **2 sockets** with **52 cores each** and SMT disabled (1 thread/core), i.e. **104 physical cores** (= 104 hardware threads) in total.

Level	Size	Cores per cache segment
L1D	48 KiB	1 (private per core)
L2	2 MiB	1 (private per core)
L3	105 MiB	52 (one shared LLC per socket)

(b) Caches and sharing (5 crd). The `likwid-topology` cache groups confirm one L1 and one L2 per core, and exactly two L3 groups (one per socket), each spanning all 52 cores of its socket.

(c) Sockets and ccNUMA domains (5 crd). The node has **2 sockets**. Sub-NUMA clustering is enabled in SNC4 mode, so each socket is split into **4 ccNUMA domains** (**8 per node**), with **13 cores per domain** (e.g. domain 0 = cores 0–12). The reported NUMA distances are 10 (local), 12 (other domain, same socket) and 21 (other socket), which reflects the on-die vs. off-socket memory access penalty.

Q2: Parallel Accumulate benchmark (40 crd)

Kernel `a[i] = a[i] + b[i]` parallelized with OpenMP and run on **one Ice Lake socket** of Fritz (2× Xeon Platinum 8360Y, 36 cores, 2.4 GHz base, SNC2 ⇒ 18 cores per ccNUMA domain). Threads were pinned with `OMP_PLACES=cores`, `OMP_PROC_BIND=close`. The benchmark harness repeats the loop until the measured time exceeds 0.2 s; a dummy parallel region is executed first to absorb the one-time pinning overhead. The clock was left at the default node setting (Turbo); since the loop is bandwidth-bound this has no measurable effect on the results. One iteration performs 1 flop and moves 24 B (load `a`, load `b`, store `a`; the write-allocate of `a` is hidden because the line is already resident from the load).

(a) Performance vs. N, 1–18 threads (20 crd). Three regimes are visible:

- **Small N ($\lesssim 10^3$):** overhead-dominated. The single-thread run is fastest here (∼5–6 Gflop/s) because the tiny array fits in L1/L2 and there is no fork/join overhead, whereas every multi-thread run pays parallel-region overhead that dwarfs the short loop.
- **Intermediate N ($\sim 10^5$–10^6):** cache-bound. The working set still fits in the aggregate L2/L3 of the socket, so performance scales with thread count and peaks at ≈60 Gflop/s (18 threads) near $N \approx 10^6$ (two arrays × 8 MB ≈ L3 capacity).
- **Large N ($\gtrsim 3 \cdot 10^6$):** memory-bound. Performance collapses onto the memory-bandwidth ceiling: ≈3.0 Gflop/s (≈73 GByte/s) for 18 threads and ≈0.87 Gflop/s (≈21 GByte/s) for a single thread. Adding threads no longer helps once the bandwidth of the ccNUMA domain is saturated.

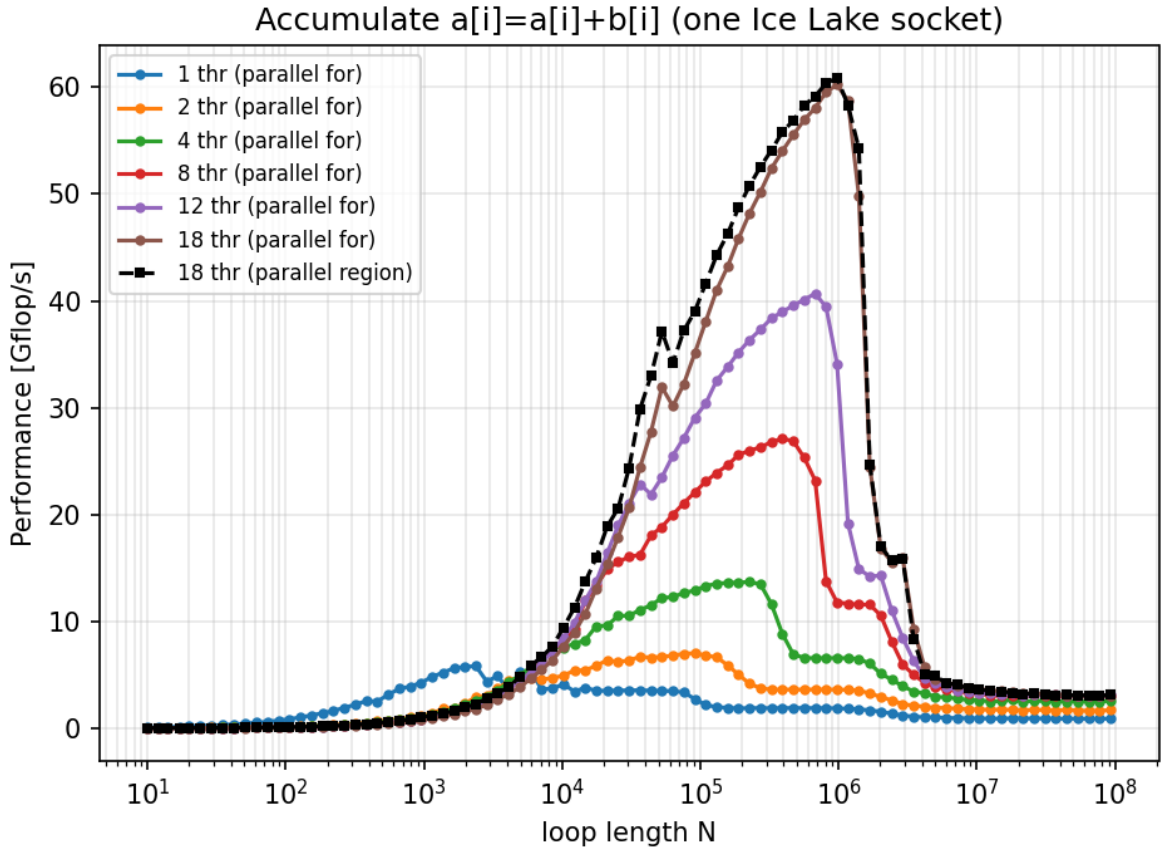


Figure 1: Performance [Gflop/s] vs. loop length N (log scale, factor 1.2 per point) for 1, 2, 4, 8, 12, 18 threads (fused `parallel for`), plus the 18-thread *single parallel region* variant of part (b) (black dashed).

(b) Repetition loop inside one parallel region, 18 threads (10 crd). The black dashed curve in the figure above is the variant where the repetition loop sits *inside* a single `#pragma omp parallel` region with a `#pragma omp for` inside. Compared with the 18-thread fused `parallel for` (brown), it is **noticeably faster in the small/intermediate-N range** and **identical at large N**.

Explanation. As shown on the "Basic OpenMP overheads" slide, a fused `parallel for` opens and closes a parallel region on *every* repetition, paying the cost of waking up and retiring the team of threads (fork/join) `NITER` times. Moving the repetition loop inside one parallel region pays this team management cost *only once*; per repetition only the lightweight `omp for` worksharing plus its implicit barrier remain. Where the loop is short (small N), this saved overhead is a large fraction of the runtime, so performance rises. At large N the kernel is memory-bound and the per-repetition overhead is negligible, so both variants coincide on the bandwidth ceiling.

(c) Bandwidth scaling on one ccNUMA domain, $N = 10^8$ (10 crd). Bandwidth rises steeply for the first few cores, then **saturates at** ≈74 GByte/s **from about 9–10 cores onward**. Measured speedup from 1 to 18 cores:

$$S = \frac{b_S(18)}{b_S(1)} = \frac{72.9\,\text{GByte/s}}{20.8\,\text{GByte/s}} \approx \mathbf{3.5}.$$

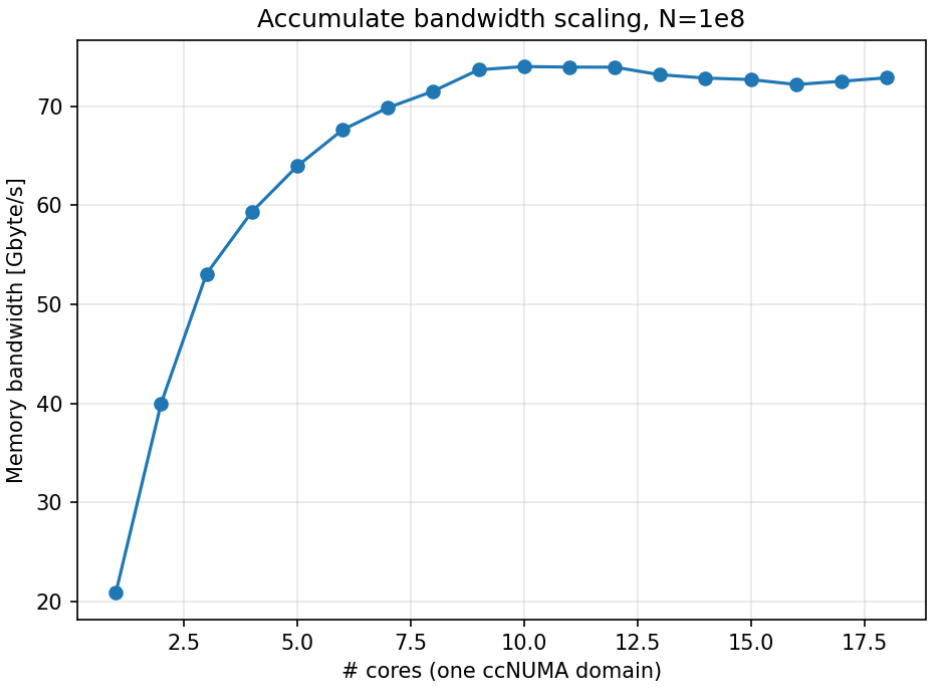


Figure 2: Memory bandwidth [GByte/s] vs. number of cores (1–18) on a single ccNUMA domain at $N = 10^8$.

This is far below the core count of 18 because memory bandwidth is a *shared* resource: a handful of cores already saturate the memory interface of the ccNUMA domain, and further cores cannot extract more bandwidth. This is the classic bandwidth-saturation behaviour of a memory-bound loop and the practical reason why thread affinity matters (keep the threads and their data on the same ccNUMA domain).

Q3: Parallel π by integration (30 crd)

The midpoint-rule integration of $\int_0^1 4\sqrt{1-x^2}\,dx = \pi$ was parallelized with `#pragma omp parallel for reduction(+:sum)`, loop length $N = 2 \cdot 10^9$, compiled *scalar* (`-O1 -no-vec`), and run with 1–36 threads on one Ice Lake socket (close pinning). All runs returned π to 13 digits, confirming correctness of the reduction. The clock was controlled with `srun --cpu-freq`.

	$T(1)$ [s]	$T(36)$ [s]	Speedup $S(36)$
Fixed 2.0 GHz	6.029	0.171	35.2
Turbo	3.443	0.108	31.9

(a)+(b) Speedup at fixed 2.0 GHz vs. Turbo (10+10 crd).

Discussion of the qualitative difference (10 crd). At **fixed** 2.0 GHz **the speedup is essentially linear** (35.2× on 36 cores). The π loop is compute-bound (a scalar `sqrt` per iteration, executed entirely in-core) with no shared bottleneck, and because every core runs at a constant 2.0 GHz regardless of how many cores are active, the per-core throughput stays constant and the work scales ideally.

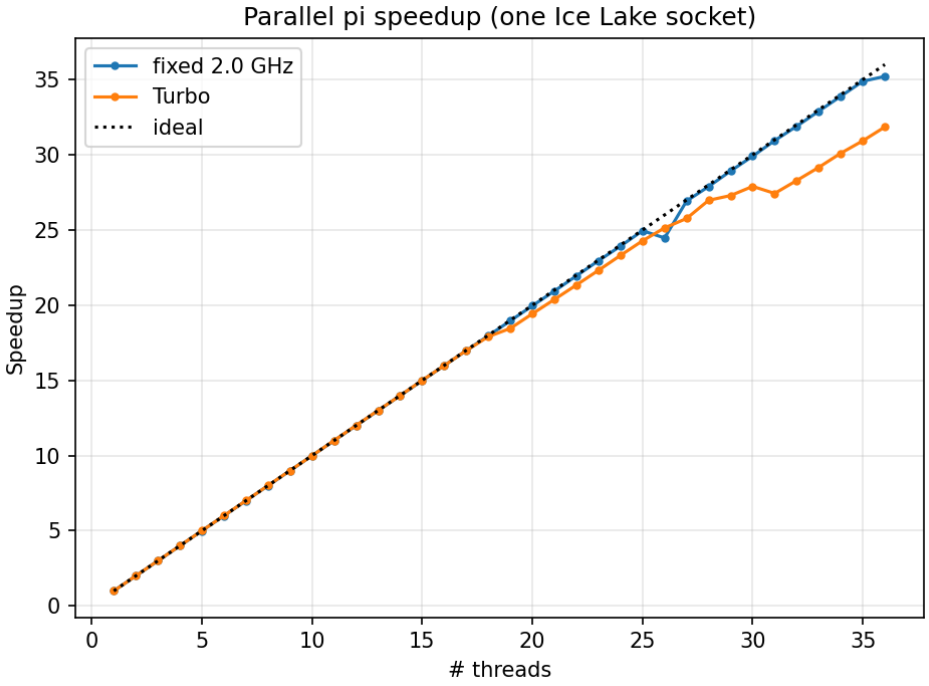


Figure 3: Speedup vs. number of threads for fixed 2.0 GHz (`--cpu-freq=2000000-2000000:performance`) and Turbo mode (`--cpu-freq=performance`); ideal scaling shown dotted.

With **Turbo enabled the speedup is sublinear** (31.9×) and bends below the ideal line beyond ∼18–20 threads. The cause is the clock frequency, not a parallel inefficiency. The processor runs at its highest Turbo clock when only a few cores are active and throttles the clock down as more cores become busy, to stay within its power/thermal budget. From the measured times the effective clock drops from ≈3.5 GHz at 1 thread ($T_{\text{turbo}}(1)/T_{2.0}(1) = 6.029/3.443 \Rightarrow 2.0 \cdot 1.75$) to ≈3.17 GHz at 36 threads ($0.171/0.108 \Rightarrow 2.0 \cdot 1.58$). Since speedup is normalised to the single-thread time of the *same* setting, and that single-thread reference enjoys the highest clock, adding cores raises the throughput by less than the core count ⇒ apparent speedup below linear. Note that Turbo is *always faster in absolute wall-clock time* (e.g. 0.108 s vs. 0.171 s at 36 threads); it only makes the speedup *metric* look worse because the 1-thread baseline is itself faster.

Q4: Resource-driven modeling (25 crd)

A loop of length N requires, per iteration, a memory transfer volume V [byte] and W instructions. The CPU has memory bandwidth b_S [byte/s], peak execution p_E [instr/cycle] and clock frequency f. The two per-iteration costs are

$$T_{\text{exec}} = \frac{W}{p_E}\ [\text{cycles}], \qquad T_{\text{data}} = \frac{V}{b_S}\ [\text{s}] = \frac{Vf}{b_S}\ [\text{cycles}].$$

(a) No overlap (10 crd). If execution and data transfer cannot overlap, the times add:

$$\boxed{T_{\text{it}} = \frac{W}{p_E} + \frac{Vf}{b_S} \quad [\text{cycles/iteration}]}$$

(b) Full overlap (5 crd). If the two resources work fully in parallel, the slower one dictates the runtime:

$$\boxed{T_{\text{it}} = \max\left(\frac{W}{p_E}, \frac{Vf}{b_S}\right) \quad \text{[cycles/iteration]}}$$

(c) Performance model (10 crd). Performance in instructions per second is $P = W/(T_{\text{it}}/f) = Wf/T_{\text{it}}$:

$$P_{\text{no-overlap}} = \frac{W}{\frac{W}{p_E f} + \frac{V}{b_S}}, \qquad P_{\text{overlap}} = \frac{W}{\max\left(\frac{W}{p_E f}, \frac{V}{b_S}\right)} \quad \text{[instr/s]}.$$

The overlap case is the Roofline form: the loop is compute-bound when $\frac{W}{p_E f} > \frac{V}{b_S}$ and memory-bound otherwise. The accumulate benchmark of Q2 is an example of the memory-bound branch (V/b_S dominates), which is why its performance saturates at the bandwidth ceiling.

Appendix A: Source code

```
/* accumulate.c  --  parallel "Accumulate" benchmark:  a[i] = a[i] + b[i]
 *
 * Build (memory-bound kernel -> SIMD ON is fine and wanted):
 *   icx -O3 -xHOST -qopenmp accumulate.c -o accumulate
 *   (or) gcc -O3 -march=native -fopenmp accumulate.c -o accumulate
 *
 * Usage:  ./accumulate <mode> [N]
 *   mode 0 = "fused"   #pragma omp parallel for            (Part a)
 *   mode 1 = repetition loop INSIDE one parallel region    (Part b)
 *   [N]    = optional single loop length.
 *            If omitted -> sweep N = 10 ... 1e8 (factor 1.2 per step).
 *
 * Output columns:  N  nthreads  Gflop/s  GByte/s  time[s]  Niter
 *   1 flop/it ; 24 byte/it (load a + load b + store a, write-allocate hidden)
 */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#ifdef _OPENMP
#include <omp.h>
#endif

static double wtime(void){
    struct timespec ts; clock_gettime(CLOCK_MONOTONIC,&ts);
    return (double)ts.tv_sec + 1.e-9*(double)ts.tv_nsec;
}

static volatile double sink = 0.0;   /* defeats dead-code elimination */

static void run(int mode, long N, double *a, double *b, long niter){
    if(mode==0){                          /* fused parallel for: fork/join EVERY k */
        for(long k=0;k<niter;++k){
            #pragma omp parallel for
            for(long i=0;i<N;++i) a[i] = a[i] + b[i];
            sink += a[k % N];             /* observable -> compiler cannot hoist  */
        }
    } else {                              /* one parallel region, omp for inside  */
        #pragma omp parallel
```

```
        {
            for(long k=0;k<niter;++k){
                #pragma omp for
                for(long i=0;i<N;++i) a[i] = a[i] + b[i];
            }
        }
        sink += a[0];
    }
}

int main(int argc,char**argv){
    int  mode    = (argc>1) ? atoi(argv[1]) : 0;
    long Nsingle = (argc>2) ? atol(argv[2]) : -1;

    int nth = 1;
    #ifdef _OPENMP
    #pragma omp parallel                 /* WARM-UP region: absorb pinning cost */
    {
        #pragma omp single
        nth = omp_get_num_threads();
    }
    #endif

    const double Nmax = 1e8, scale = 1.2;
    double Nf = (Nsingle>0) ? (double)Nsingle : 10.0;

    printf("# N  nthreads  Gflop/s  GByte/s  time  Niter   (mode=%d)\n", mode);

    do {
        long N = (long)Nf;
        double *a = (double*)malloc((size_t)N*sizeof(double));
        double *b = (double*)malloc((size_t)N*sizeof(double));
        if(!a||!b){ fprintf(stderr,"alloc fail at N=%ld\n",N); return 1; }

        #pragma omp parallel for           /* NUMA-aware first touch (static)  */
        for(long i=0;i<N;++i){ a[i]=1.0; b[i]=2.0; }

        long niter = 1; double t = 0.0;
        do {                               /* harness: grow niter until >0.2 s  */
            niter *= 2;
            double s = wtime();
            run(mode,N,a,b,niter);
            t = wtime()-s;
        } while(t < 0.2);

        double flop = (double)N*(double)niter;          /* 1 flop / iter        */
        double byte = 24.0*(double)N*(double)niter;     /* 24 byte / iter       */
        printf("%ld %d %.4f %.4f %.4f %ld\n",
               N, nth, flop/t/1e9, byte/t/1e9, t, niter);

        free(a); free(b);
        Nf *= scale;
    } while(Nsingle<0 && Nf <= Nmax*1.0001);

    return 0;
}
```

Listing 1: accumulate.c

```c
/* integrate_omp.c  --  parallel "pi by integration" (OpenMP reduction)
 *
 * Build (SCALAR code, no SIMD -- as required by the assignment):
 *   icx -O1 -no-vec -qopenmp integrate_omp.c -o integrate_omp -lm
 *   (gcc) gcc -O1 -fno-tree-vectorize -fopenmp integrate_omp.c -o integrate_omp -lm
 *
 * Output:  nthreads  time[s]  pi
 */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#ifdef _OPENMP
#include <omp.h>
#endif

static double wtime(void){
    struct timespec ts; clock_gettime(CLOCK_MONOTONIC,&ts);
    return (double)ts.tv_sec + 1.e-9*(double)ts.tv_nsec;
}

int main(void){
    const long   N  = 2000000000L;        /* 2 x 10^9 */
    const double dx = 1.0/(double)N;
    double sum = 0.0;
    int nth = 1;

    #ifdef _OPENMP
    #pragma omp parallel                   /* WARM-UP region: absorb pinning cost */
    {
        #pragma omp single
        nth = omp_get_num_threads();
    }
    #endif

    double s = wtime();
    #pragma omp parallel for reduction(+:sum)
    for(long i=0;i<N;++i){
        double x = ((double)i + 0.5)*dx;
        sum += 4.0*sqrt(1.0 - x*x);
    }
    double pi = sum*dx;
    double t  = wtime()-s;

    printf("%d %.6f %.15f\n", nth, t, pi);
    return 0;
}
```

Listing 2: integrate_omp.c

Appendix B: Compilation and execution

```
# Accumulate (memory-bound -> SIMD on), Ice Lake, default partition:
icx -O3 -xHOST -qopenmp accumulate.c -o accumulate
export OMP_PLACES=cores OMP_PROC_BIND=close
```

```
for T in 1 2 4 8 12 18; do OMP_NUM_THREADS=$T ./accumulate 0 > acc_a_${T}.dat; done
OMP_NUM_THREADS=18 ./accumulate 1 > acc_b_18.dat
for T in $(seq 1 18); do OMP_NUM_THREADS=$T ./accumulate 0 100000000 \
    | grep -v '^#' >> acc_c.dat; done

# Parallel pi (scalar), one socket, clock controlled via srun:
icx -O1 -no-vec -qopenmp integrate_omp.c -o integrate_omp -lm
for T in $(seq 1 36); do OMP_NUM_THREADS=$T \
    srun --cpu-freq=2000000-2000000:performance --cpu-bind=none \
    ./integrate_omp >> pi_fixed2.0.dat; done           # (a) fixed 2.0 GHz
for T in $(seq 1 36); do OMP_NUM_THREADS=$T \
    srun --cpu-freq=performance --cpu-bind=none \
    ./integrate_omp >> pi_turbo.dat; done              # (b) Turbo
```